

algoritma analizi

Asymptotic Analysis : Order of Growth $n \rightarrow \infty$

$n = 100$ milyon	$\lg_2 n$	n	$n \cdot \lg_2 n$	n^2	$n = 100$	2^n
	0.027 MS	0.1 sec	2.66 sec	115.7 day		$4 \cdot 10^3$ year

→ Algoritmayı kullanmak mantıklı mı?

→ Bu çalışan algoritmayı gerçekte kullanabilecek miyiz?

$\equiv$
 for ($i=0$; $i < n$; $i++$) {
 assignment = 1
 for ($j=0$
 less than compare: $n+1$
 for ($j=0$
 equal to compare: n
 array access: n
 count ++;
 increment: $n+2n$
 }
 }
 $4n+2 \approx 5n+2$
 $n \rightarrow \infty$

karmaşıklık buna göre hesaplanır

① ignore constant terms ($c \rightarrow 10^{10}$ olursa çok değişir)

$$4n^2 + 5n + 9$$

② ignore coefficients ($m \times n$) (m çok büyük değilken)

$$4n^2 + 5n \rightarrow n^2 + n$$

③ pick the most significant term

$$n^2 + n \rightarrow \underline{n^2}$$

Asymptotic Notation $n \rightarrow \infty$

$O()$: Big O $\Theta()$: Big-Theta $\Omega()$: Big-Omega

Asymptotic Upper Bound Asymptotic Tight Bound Asymptotic Lower Bound

① Big - Oh Notation

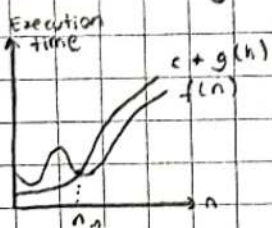
$$f(n) = 10n + 30 \xrightarrow{?} O(n)$$

$g(n) = n$

$$f(n) \in O(g(n))$$

If $0 \leq f(n) \leq c \cdot g(n)$ for all $c > 0$ (constant), $n \geq n_0 > n$

then $O(g(n))$: upper bound on the growth of a function $f(n)$



$$0 \leq 10n + 30 \leq c \cdot g(n)$$

$$0 \leq 10n + 30 \leq c \cdot n$$

$$30 \leq (c - 10) \cdot n$$

$$f(n) \in O(n) \checkmark$$

$$f(n) = 10n + 30 \xrightarrow{?} O(n^2)$$

$$10n + 30 \leq c \cdot n^2 \checkmark$$

$$\frac{30}{c-10} \leq n$$

$$30 \leq n$$

$$c = 11$$

$$n_0 = 30$$

herhangi bir
sabit
uygun olan
bul
 $c = 11$
 $n_0 = 30$ olmaz

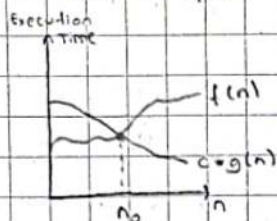
→ olabilir ama yapacağımız
sey olabilecek en küçük
upper bound'u bulmak

$$f(n) = n^2 + n \xrightarrow{?} O(n) \quad n^2 + n \leq c \cdot n$$

$$g(n) = n \quad \times \quad 0 \leq (c - n - 1) \cdot n$$

BIG OMEGA NOTATION : Asymptotic Lower Bound

If $f(n) \geq c \cdot g(n) \geq 0$ for all $c > 0$ $n \geq n_0 > 0$
 then $f(n) \in \Omega(g(n))$ $\rightarrow$ asymptotic lower bound of $f(n)$



$$5n^2 \in \Omega(n)$$

$$5n^2 \geq n \cdot c \geq 0$$

$$n_0 = 1$$

$$5 \geq c \quad c = 3$$

$$n_0 = 1$$

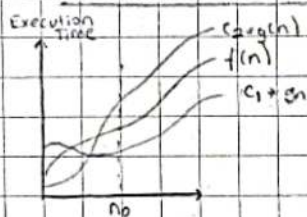
$$4n^2 \in \Omega(n^2)$$

$$4n^2 \geq c \cdot n^2 \cdot g(n)$$

$$n^2 \cdot (4 - c) \geq 0 \quad c = 3$$

$$n^2 \geq 0 \quad n_0 = 1$$

BIG THETA NOTATION : Asymptotic Tight Bound



$$\text{If } 0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$$

$$\text{for all } c_1 < c_2 \quad n \geq n_0 > 0$$

$$c_1 > 0, c_2 > 0$$

$$\text{then } f(n) \in \Theta(g(n))$$

$f(n)$ is asymptotically equal to $g(n)$

$$\left. \begin{array}{l} f(n) \in O(g(n)) \\ f(n) \in \Omega(g(n)) \end{array} \right\} \Theta(g(n))$$

① $f(n) = 2n + 5 \quad ? \quad \Theta(n)$

$$f(n) \in ? O(n)$$

$$2n + 5 \leq c_1 \cdot n$$

$$5 \leq (c_1 - 2) \cdot n$$

$$c_1 = 7$$

$$n_0 = 1$$

② $f(n) \in ? \Omega(n)$

$$2n + 5 \geq c_2 \cdot n$$

$$2n + 5 \geq c_2 \cdot n$$

$$5 \geq (c_2 - 2) \cdot n$$

$$c_2 = 1$$

$$n_0 = 1$$

$$7 \geq 1 \quad \checkmark$$

$$n_0 = 1 \quad \checkmark$$

ayni kalmali

3. HAFTA

$$\sum_{i=1}^n 1 = n$$

$$\sum_{i=1}^n i = 1 + 2 + 3 + \dots + n = \frac{n \cdot (n+1)}{2}$$

$$\sum_{i=1}^n c = n \cdot c$$

$$\sum_{i=1}^n i^2 = 1^2 + 2^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6} = \frac{n^3}{3}$$

① Mathematical Analysis of Non-Recursive Algorithms

```
int findMax (int n, int A[])
```

```
{
```

```
    int i, max;
```

```
    max = A[0];
```

```
    for (i = 1; i < n; i++)
```

```
        if (A[i] > max)
```

```
            max = A[i];
```

```
    return max;
```

```
}
```

BASIC
OPERATION!

① Identify algorithm's basic operation

② Determine worst, average, best case complexity for input size of n

$$c(n) = \sum_{i=1}^{n-1} 1 = n - 1 \rightarrow n$$

of comparisons

$$c_{\text{best}}(n) = n$$

$$c_{\text{worst}}(n) = n$$

$$c_{\text{average}}(n) = n$$

for random inputs

$$O(n) \quad \Omega(n) \quad \Theta(n)$$

$$\left. \begin{array}{l} c_{\text{best}}(n) = n \\ c_{\text{worst}}(n) = n \\ c_{\text{average}}(n) = n \end{array} \right\} \Theta(n)$$

② Sequential Search

```

int sequentialSearch (int n, int a[], int x) {
    int i = 0;
    while ((i < n) && (a[i] != x)) {
        i++;
    }
    if (i < n)
        return i;
    else
        return -1;
}
    
```

BASIC
OPERATION!

$$\begin{aligned}
 C_{\text{best}}(n) &= 1 \\
 C_{\text{worst}}(n) &= n \\
 C_{\text{avg}}(n) &= n
 \end{aligned}$$

Tamamı için
 Θ kullanırsın

Tamamı için
 $\Theta(n)$
 $\Omega(1)$

$p: 0 \leq p \leq 1$ successful search
probability of i^{th} position: $\frac{p}{n}$
 $1-p$: unsuccessful

$$C_{\text{avg}}(n) = \left[1 \cdot \frac{p}{n} + 2 \cdot \frac{p}{n} + \dots + n \cdot \frac{p}{n} \right] + n \cdot (1-p)$$

number of comparison successful search # of comp. unsuccessful search

$$C_{\text{avg}}(n) \approx O(n)$$

just similar

$$C_{\text{avg}}(n) = \Theta(n) \rightarrow \text{bunu da görebilirsin}$$

$$= \frac{p}{n} \left[\frac{1+2+\dots+n}{2} \right] = \frac{p(n+1)}{2} + n(1-p)$$

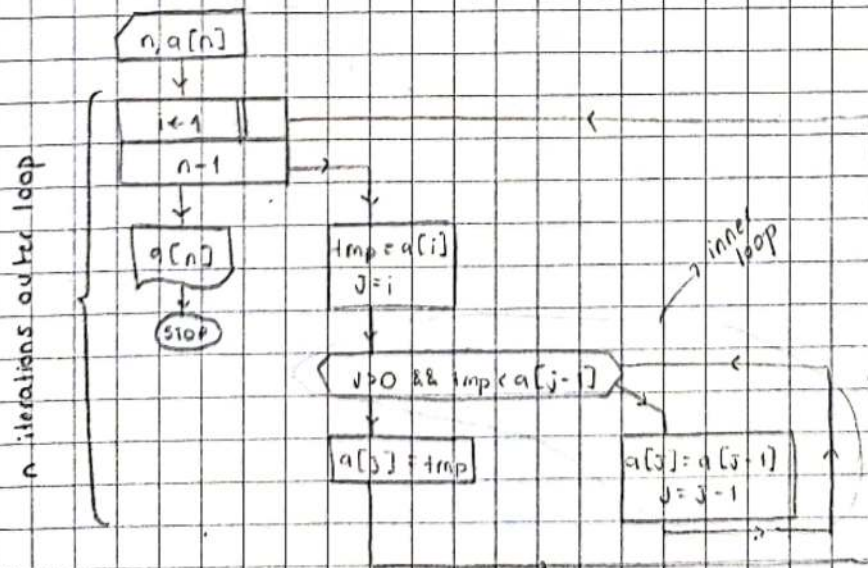
$p=1$ successful search $\rightarrow \frac{n+1}{2} + 0 = \frac{n+1}{2}$

eleman
dizide

$p=0$ unsuccessful search $\rightarrow 0 + n \cdot (1-0) = n$
dizide değil
 $C_{\text{avg}}(n) = n$

Insertion Sort

9 3 7 1 4 5
3 9 7 1 4 5
3 7 9 1 4 5
1 3 7 9 4 5



Best case: sorted array $= n-1 \times n$

1 3 5 7 9
başlı başı karıştırmaz

Worst case: descending order
sorted array

8 5 2 1

Worst case: $i=1 \rightarrow 1$
 $i=2 \rightarrow 2$

$i=n \rightarrow n$

$$\sum_{i=1}^{n-1} i = \frac{(n-1)(n)}{2} \sim \frac{n^2}{2}$$

from k^{th} position to j^{th}
of comp. $\rightarrow k+j+1$ for one element

inner loop:

$$\frac{1}{k} \sum_{j=1}^k (k-j+1) = \frac{1}{k} \left(k^2 - \frac{k(k+1)}{2} + k \right) = \frac{k^2 + k}{2k} = \frac{k+1}{2}$$

for inner loop

Average cost for N elements:

$$\sum_{k=1}^N \frac{k+1}{2} = \frac{1}{2} \left(\frac{(N-1) \times N}{2} + \frac{N+1}{2} \right) \sim \frac{N^2}{4}$$

time complexity of insertion sort: $O(n^2)$

average case complexity: $O(n^2)$
 $\Theta(n^2)$

$\sim \frac{n^2}{4}$

③ Mathematical Analysis of Recursive Algorithms

1) Factorial Function

$$0! = 1 \quad n! = 1 \cdot 2 \cdot \dots \cdot n$$

$$n! = n \times (n-1)!$$

$$f(n) = \begin{cases} 1 & n=0 \\ n \cdot f(n-1) & n>0 \end{cases}$$

```
int factorial (int n) {
```

```
    if (n == 0)
```

```
        return 1;
```

```
    else
```

```
        return (n * factorial (n-1));
```

BASIC
OPERATION

```
}
```

→ recursive description of the function
Recurrence Relation

$$M(n) = M(n-1) + 1 \quad n > 0$$

number of multiplication $M(0) = 0 \quad n = 0$

1. Identify the basic operation

2. Setup a recurrence relation for recursive operation

a) Backward Substitution

$$M(n) = M(n-1) + 1$$

$$= M(n-2) + 1 + 1 = M(n-2) + 2$$

$$= M(n-3) + 1 + 2 = M(n-3) + 3$$

$$i^{\text{th}} \text{ Step: } M(n-i) + i$$

for $n=i$

$$= M(i-i) + i \Rightarrow M(n) = n \rightarrow \text{Best case} = \text{worst case} = \text{average case} \Rightarrow \Theta(n)$$

Find the number of binary digits in the binary representation of a positive decimal integer.

$$\begin{array}{r} 27 \div 2 \\ 26 \div 2 \\ 13 \div 2 \\ 12 \div 2 \\ 6 \div 2 \\ 3 \div 2 \\ 2 \div 2 \\ 1 \div 2 \\ 0 \end{array} \rightarrow 11011$$

5 digit

```
int binaryDigits (int n) {
```

```
    if (n == 1)
```

```
        return 1;
```

```
    else
```

```
        return 1 + binaryDigits (n/2);
```

BASIC
OPERATION

```
}
```

Recurrence Relation

$$A(n) = A(n/2) + 1 \quad n > 1$$

$$A(1) = 0 \quad n = 1$$

$$A(n) \in \Theta(\log_2 n)$$

$$A(n) = A(n/2) + 1$$

$$= A(n/4) + 1 + 1 = A(n/4) + 2$$

$$= A(n/8) + 1 + 2 = A(n/8) + 3$$

$$i^{\text{th}} \text{ step} \Rightarrow A(n/2^i) + i$$

$$\text{for } n=2^i \rightarrow A(2^i/2^i) + i$$

$$A(1) + i$$

$$0 + i$$

$$A(n) = i \quad n = 2^i \Rightarrow i = \log_2 n$$

$$A(n) = \log_2 n$$

b) Forward Substitution

$$T(n) = \begin{cases} 1 & \text{if } n=1 \\ T(n-1) + 1 & \text{otherwise} \end{cases}$$

n T(n)

1 1

2 T(2-1) + 1 = 2

3 T(3-1) + 1 = 3

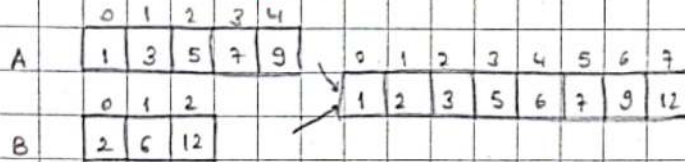
i

n T(n-1) + 1 = n

10^8 compares / seconds

n	1000	1,000,000	1,000,000,000
insertion sort ($O(n^2)$)	instant	2.3 h	317 years
merge sort ($O(n \log_2 n)$)	instant	1. sec	18 min
quick sort ($O(n \log_2 n)$)	instant	0.6 sec	12 min

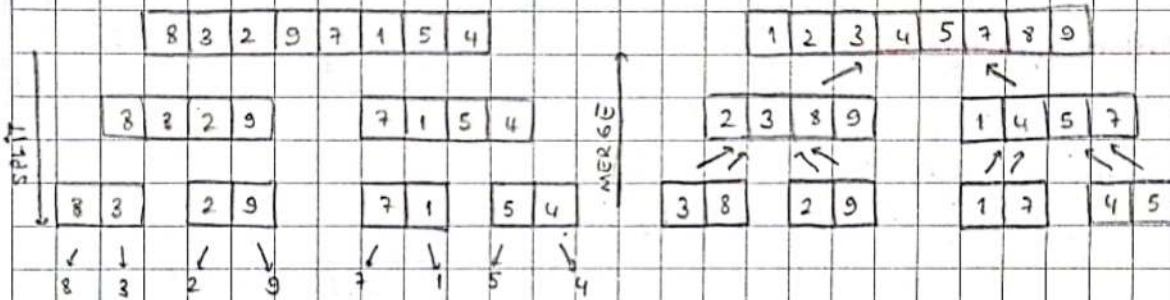
Divide and Conquer



→ arkasına ekleyip sort uygularsak → $O(n^2)$

→ sıralı olmalarından faydalanıp baston karşılaştırıp sıralarsak → $O(n)$

→ sırasız iki diziye

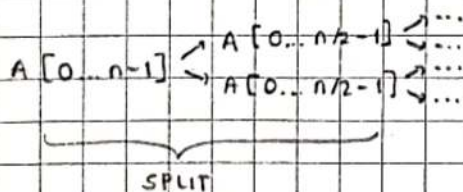


Divide and Conquer

1. DIVIDE the problem into smaller problems
2. CONQUER by solving these problems
3. COMBINE these results together

Merge Sort

1. SPLIT the array
2. SORT each subsequence



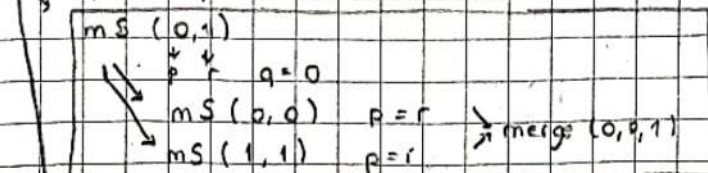
```

void mergeSort (int A[], int p, int r) {
    if (p < r) {
        q = (p+r)/2;
        mergeSort (A, p, q);
        mergeSort (A, q+1, r);
        merge (A, p, q, r);
    }
}
    
```

üstteki dizi için $p=0, r=7, q=3$

$MS(0, 3)$

$q = 3/2 = 1$



→ aynı şey 2 ve 3 için olacak

merge(0, 1, 3) elinde 2, 3, 8, 9 dizisi olacak


```
void merge (int A[], int p, int q, int r) {
```

```
    int i, j, k;
```

```
    int *B;
```

```
    n = (r - p + 1) → # of elements
```

```
    B = (int *) malloc...
```

```
    i = k; k = 0;
```

```
    j = q + 1; → initial i+1
```

```
    while ((i <= q) && (j <= r)) {
```

```
        if (A[i] < A[j])
```

```
            B[k++] = A[i++];
```

```
        else
```

```
            B[k++] = A[j++];
```

```
    }
```

```
    while (i <= q)
```

```
        B[k++] = A[i++];
```

```
    while (j <= r)
```

```
        B[k++] = A[j++];
```

```
    for (i = p, k = 0; i <= r; i++, k++)
```

```
        A[i] = B[k];
```

```
    free (B);
```

```
}
```

Recurrence Relation of Merge Sort

$$T(n) = \begin{cases} O(1) & n=0,1 \\ 2 \cdot T(n/2) + n & \text{if } n > 1 \end{cases}$$

$\underbrace{2 \cdot T(n/2)}_{\text{2 merge sort}} \quad \underbrace{+ n}_{\text{merge}}$

Backwards Substitution

$$T(n) = 2 \cdot T(n/2) + n \quad n > 1$$

$$= 2 \cdot (2 \cdot T(n/4) + n/2) + n = 4T(n/4) + 2n$$

$$= 4 \cdot (2 \cdot T(n/8) + n/4) + 2n = 8T(n/8) + 3n = 2^i \cdot T(n/2^i) + i \cdot n$$

$$= n + i \cdot n$$

$$= n + \log_2 n \cdot n \rightarrow O(n \cdot \log_2 n)$$

Best = worst = avg

$$\Theta(n \cdot \log_2 n)$$

Master Theorem

General divide and conquer recurrence

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n) \quad T(1) = c$$

of subproblems that are solved recursively

size of each subproblem

cost of combining

constant

$$T(n) = 2^a \cdot T(n/2^b) + n \quad \text{if } f(n) \text{ merge}$$

If $f(n) \in \Theta(n^\alpha)$ when $\alpha \geq 0$

$$a=2 \quad b=2 \quad f(n) = n^1 \rightarrow \alpha=1$$

$$2 = 2^1 \quad a = b^1$$

$$T(n) \in \Theta(n^1 \cdot \log_2 n)$$

$$T(n) = \begin{cases} \Theta(n^\alpha) & \text{if } a < b^\alpha \\ \Theta(n^\alpha \log_2 n) & \text{if } a = b^\alpha \\ \Theta(n^{\log_b a}) & \text{if } a > b^\alpha \end{cases}$$

Aynı şey 0 ve α için de geçerli!

example

$$T(n) = T\left(\frac{2}{3}n\right) + 1 \rightarrow n^0$$

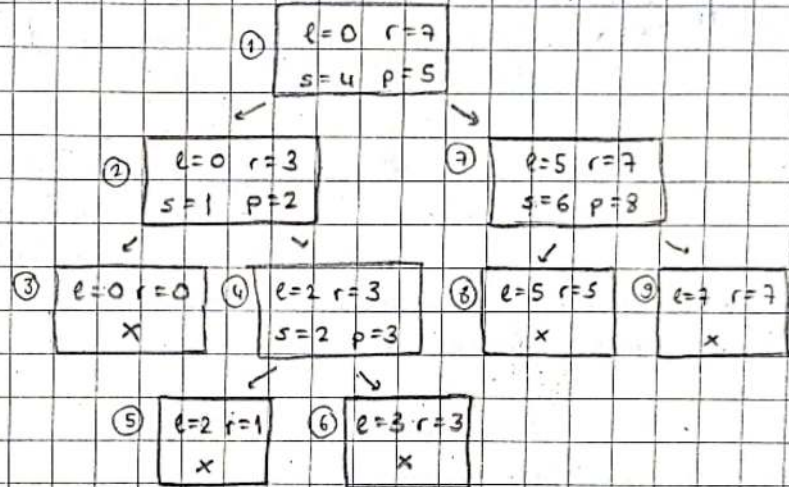
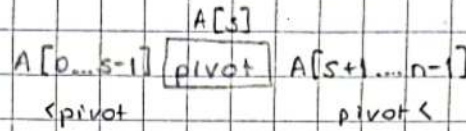
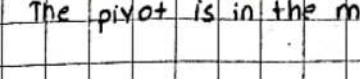
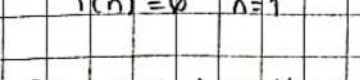
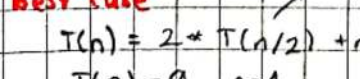
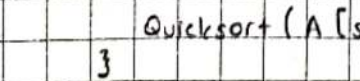
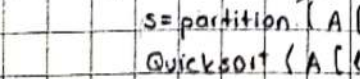
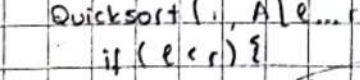
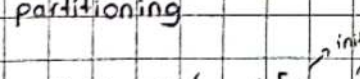
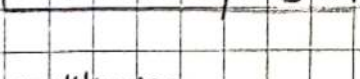
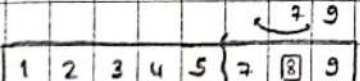
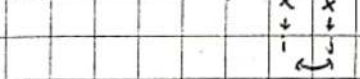
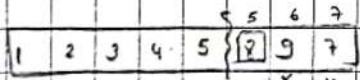
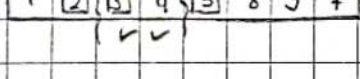
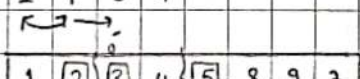
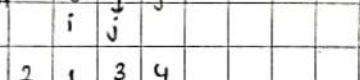
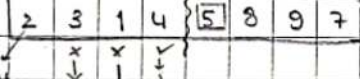
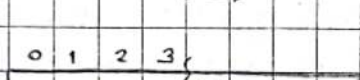
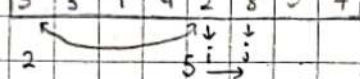
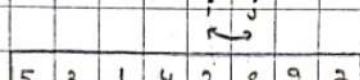
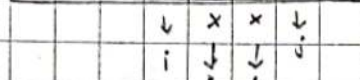
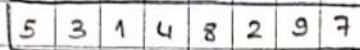
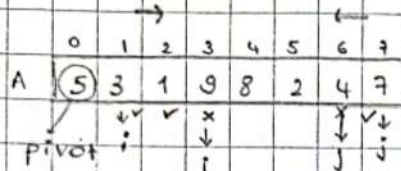
$$a=1 \quad b=\frac{3}{2} \quad \alpha=0$$

$$1 = \frac{3}{2}^0 \rightarrow 1=1$$

$$T(n) \in O(n^0 + \log_2 n) \in O(\log_2 n)$$

Quick Sort

1962 Tony Hoare



partitioning

QuickSort($l, A[l \dots r]$) {
 if ($l < r$) {
 $s = \text{partition}(A[l \dots r]);$
 QuickSort($A[l \dots s-1]$);
 QuickSort($A[s+1 \dots r]$);
 }

pivot
 gerine yerleşir
 küçükler sağına
 büyükler soluna

Best Case

$$T(n) = 2 * T(n/2) + n$$

$$T(n) = 0 \quad n=1$$

Quick sort
 partitioning
 $n \geq 1$

$$T(n) \in \Theta(n * \log_2 n)$$

BEST CASE

The pivot is in the middle at each partition step!

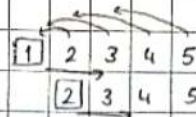
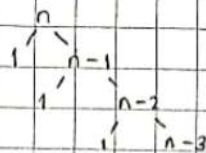
Worst Case

$$A[0] < A[1] < \dots < A[n-1]$$

$$T(n) = T(n-1) + N \rightarrow \text{partitioning}$$

$$T(0) = T(1) = 0$$

$$T_{\text{worst}}(n) \in \Theta(n^2)$$



Average Case

position of pivot	left of pivot	right of pivot	$T(n_{\text{left}})$	$T(n_{\text{right}})$
0	0	$n-1$	$T(0)$	$T(n-1)$
1	1	$n-2$	$T(1)$	$T(n-2)$
2	2	$n-3$	$T(2)$	$T(n-3)$
$\vdots$				
$n-1$	$n-1$	0	$T(n-1)$	$T(0)$

$$T(n) = \frac{1}{n} \sum_{1 \leq k \leq n} T(k-1) + T(n-k) + n$$

left subarray
right subarray
partitioning

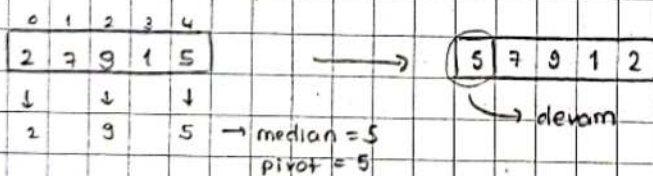
$$T(n) = 1.39 n \log_2 n \in O(n \log_2 n) \rightarrow \text{merge sorttan \%39 fazla kar\u015fıla\u015ftırma ama daha az yer de\u011fi\u015ftirme oldu\u011fu i\u00e7in daha hızlı!}$$

QUICK SORT $\rightarrow$ IN-PLACE (azıradan yere gerek yok)

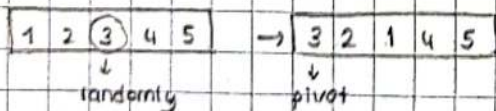
MERGE SORT $\rightarrow$ NOT IN-PLACE X

Worst case'i engellemek i\u00e7in

- ① Shuffle the array
- ② median of samples



- ③ Randomly pick an element as pivot



Decrease And Conquer

- ① Decrease by a constant factor

binary search

problem of size n

$\downarrow$

problem of size $n/2$

$\downarrow$

$\vdots$

solution to the original problem

$$T(n) = T(n/2) + c$$

② Variable size decrease

$$\text{GCD}(a, b) = \begin{cases} a & \text{if } b = 0 \\ \text{GCD}(b, a \bmod b) & \text{otherwise} \end{cases}$$

$$\text{GCD}(80, 44)$$

$$\hookrightarrow \text{GCD}(44, 36)$$

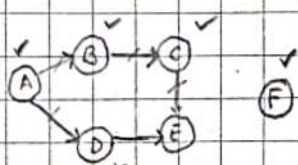
$$\hookrightarrow \text{GCD}(36, 8)$$

$$\hookrightarrow \text{GCD}(8, 4)$$

$$\hookrightarrow \text{GCD}(4, 0) = 4 //$$

③ Decrease by a constant

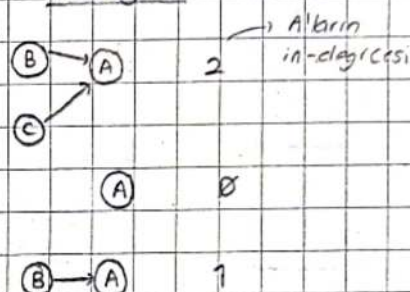
Topological Sort



Digraph with no DIRECTED CYCLE

$A, F \rightarrow B, D \rightarrow C \rightarrow E$

in-degree



in degree	adjacency list
0	A → B → D
1	B → C
1	C → E
1	D → E
2	E → NULL
0	F → NULL

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F$

→ Tek bir çözüm o yok

① Store each vertex's in-degree in an array $O(|E|)$

② While there are vertices remainly:

- Find a vertex with in degree zero $O(|V|^2)$

- Reduce in degree of all vertices adjacent to it by 1 $O(|E|)$

- Mark this vertex's in-degree -1

$O(|V|^2 + |E|)$

using Queue

① Store

② Initialize a queue with all in-degree zero vertices $O(|V|)$

③ While there are vertices remaining in the queue

- Dequeue and output $O(|V|)$

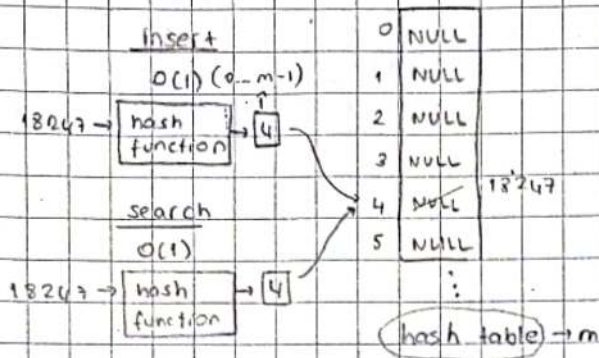
- Reduce in-degree of all vertices adjacent to it by 1 $O(|E|)$

- ENQUEUE vertices with in-degree zero $O(|E|)$

$O(|V| + |E|)$

	worst case			average case		
	search	insert	delete	search	insert	delete
sequential search	n	1	n	n/2	1	n/2
binary search	lg n	n	n	lg n	n/2	n/2
BST	n	n	n	lg n	lg n	lg n
balanced tree	lg n	lg n	lg n	lg n	lg n	lg n

Hashing



- ① How big should the table be?
- ② How to select a hash table?
- ③ How to solve collision?

Hash Functions

① Division method

$$\text{hashfunction}(\text{key}) = \text{key} \bmod m$$

table size $0 \rightarrow m-1$

table size = prime number

çakışmayı azaltır.

0	?
1	
2	
3	
4	25
5	
6	

key = 25

$$\text{hash}(25) = 25 \% 7 = 4$$

$$\text{hash}(7) = 7 \% 7 = 0$$

② Multiplication Method

$$\text{hashfunction}(\text{key}) = \left[m * \left[(\text{key} * A) \bmod 1 \right] \right]$$

Knuth

$$A = \frac{\sqrt{5}-1}{2} = 0.618033$$

m = 10.000 key = 123456 → tabloya yerleşecek

$$\text{hashfunction}(123456) = 10.000 * \left[123456 * A \bmod 1 \right]$$

$$76300.0001151$$

$$0.0001151 * 10.000 \rightarrow [11.51] = 11$$

COLLISION PROBLEM

① Open Addressing (IBM - 1953) → table size = prime number

1.1) Linear Probing

↳ each check is a probe!

INSERT

$$\text{hashF}(\text{key}, i) = [\text{hashF}(\text{key}) + i] \% m$$

key = 16

i = 0 → $\text{hash}(16, 0) = 16 \% 11 = 5$ dolu

i = 1 → $\text{hash}(16, 1) = (5+1) \% 11 = 6$ ✓

key = 20

i = 0 → $20 \% 11 = 9$ x

i = 1 → $(9+1) \% 11 = 10$ x

i = 2 → $(9+2) \% 11 = 0$ ✓

0	20
1	
2	
3	
4	
5	8
6	16
7	
8	
9	9
10	21

m = 11

key = 5

$$\text{hash}(\text{key}) = 5 \% 11$$

key = 9

$$\text{hash}(\text{key}) = 9 \% 11$$

key = 16

$$\text{hash}(\text{key}) = 16 \% 11$$

M ≠ N olmalı

çakışma yok diller

SEARCH

key = 10

$$10 \% 11 = 10$$

$$i=0 \rightarrow 10 \neq 21$$

$$i=1 \rightarrow (10+1) \% 11 \rightarrow 0 \quad 10 \neq 20$$

$$i=2 \rightarrow (10+2) \% 11 \rightarrow 1 \rightarrow \text{NULL} \Rightarrow 10 \text{ sayısı bulunamadı!}$$

$i < m$ kontrolü ile başlangıç noktasına geri dönüp dönülmediği kontrol edilmelidir.

(hash[adr] != NULL) && (i < m) && (hash[adr] != key) {
...
}

→ Clustering Problem → arka arkaya sürekli bir kayma oluşuyor.

1.2) Double Hashing

$$\text{hashF}(\text{key}, i) = [\text{hashF1}(\text{key}) + i * \text{hashF2}(\text{key})] \% m$$

i^{th} check offset

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	

$$\text{hash1}(\text{key}) = \text{key} \% 13$$

$$\text{hash2}(\text{key}) = \text{key} \% 11$$

$$\text{key} = 14 \quad i=0 \rightarrow \text{hash1} = 1$$

$$\text{key} = 79 \quad i=0 \rightarrow \text{hash1} = 1 \times$$

$$i=1 \rightarrow \text{hash}(79) = [1 + 1 * (79 \% 11)] \% 13 = 3 \quad \checkmark$$

table size = 13

Birthday Paradox: 2 kişilik doğum gününün farklı olma olasılığı

$$\frac{365}{365} \times \frac{364}{365} \quad k \text{ kişinin farklı olma olasılığı: } \frac{365}{365} \times \frac{364}{365} \times \dots \times \frac{(365) - (k-1)}{365}$$

table size: m

$$2 \text{ farklı key adr. olma olasılığı: } \frac{m}{m} \times \frac{m-1}{m} = \frac{m-1}{m}$$

$$k \text{ farklı key " " " : } \frac{m-1}{m} \times \dots \times \frac{m-(k-1)}{m}$$

using Taylor expansion: Approximately

$$\text{collision probability: } 1 - e^{-\frac{(k-1) \times k}{2m}}$$

$m = 2^{32}$ table size → 77163 insert sonrası çakışma olasılığı % 50

250.000 insert sonrası çakışma " " % 100

$$\text{LOAD FACTOR: } \frac{N}{M} = \frac{10}{100} \% 10$$

$$p(\text{hits occupied cell}) = \alpha$$

$$p(\text{hits empty cell}) = 1 - \alpha$$

$$p(\text{second hit is empty}) = \alpha * (1 - \alpha)$$

$$p(k^{\text{th}} \text{ hit is empty}) = \alpha^{k-1} * (1 - \alpha)$$

Σ

expected time for successful search $O(1 + \alpha)$

α normalde 0,5 kabul edilir.

$$n = 50 \quad m = 100$$

INSERTION

$n=10$ $m=100$	α	# of probes (adim)
	0,1	1,11
$n=20$ $m=100$	0,2	1,28
	0,3	1,52
	0,4	1,79
	0,5	2,5
	0,6	3,62
	0,7	6,06
	0,8	13
	0,9	50,5

table size $m=8$

keys: 64, 100, 128, 200, 300, 400, 500

$x \bmod 8 \rightarrow 64, 128, 200, 400 \rightarrow 0^{\text{th}} \text{ adr}$

$100, 300, 500 \rightarrow 4^{\text{th}} \text{ adr}$

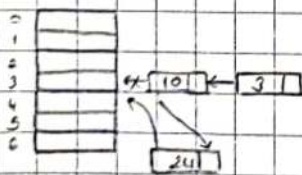
$$\text{GCD}(64, 100, 128, 200, 300, 400, 500, (8)) = 4$$

$m=11$

$$\text{GCD}(64, 100, 128, 200, 300, 400, 500, (11)) = 1$$

$$\begin{matrix} \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow \\ 9 & 1 & 7 & 2 & 3 & 4 & 5 \end{matrix}$$

② Separate Chaining $M \sim N/5$ olunc. $M < N$



$$\text{key} = 3 \quad 3 \% 7 = 3$$

$$\text{key} = 10 \quad 10 \% 7 = 3 \rightarrow \text{baza yerleştirmek daha kolay}$$

$$\text{key} = 24 \quad 24 \% 7 = 3$$

```
struct node {
    int key;
    char name[40];
    struct node * next;
};
```

```
struct hash {
    struct node * head;
    int count;
};
```

struct hash * hashTable = NULL;

key: UNIQUE TCN, studentID, phoneNo, clientID

① String

$$\text{"Elif"} \quad \text{hash("Elif")} \xrightarrow{HF} \text{adr} \quad 'E' + 'L' + 'I' + 'F' = 292 \quad 292 \% m$$

mantıklı değil
File de kullanılabilir

length = 1

$$\text{key} = K^{e-1} * \text{str}[0] + K^{e-2} * \text{str}[1] + \dots + K^0 * \text{str}[e-1]$$

↓
int karsılığı
small prime number

Horner's method $\rightarrow K=31$

length = strlen(s);

```
for (i=0; i < length; i++) {  
    tmp = s[i] - 'A' + 1;  
    hashKey = K * hashKey + tmp;  
}
```

Universal Hashing

$h_1()$
:
 $h_k(i)$ } randomly choose
a hash function!

$$h_{a,b}(key) = [(a * k + b) \bmod p] \bmod m$$

a, b, p değişken

h1 için a=5 b=4 p=21

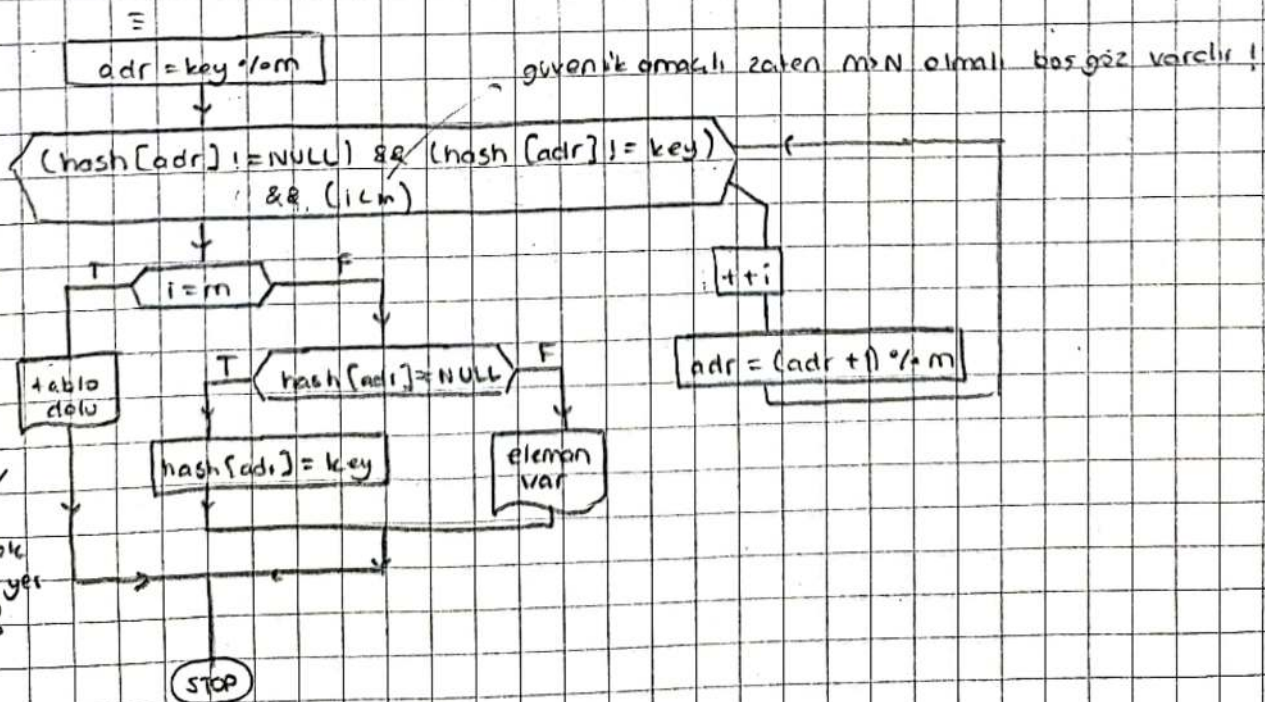
h2 için a=7 b=3 p=27

Open Addressing'te

Silme işlemi yaparken
varolan elemanların önünde flag
değeri tutulur 1 var 0 silindi

→ çünkü NULL olursa arama işlemi
bozulur.

Soru: bir hash tablosuna yeni bir eleman eklenmek isteniyor. Eğer bu eleman tabloda
daha önce varsa eklemeyen yoksa ekleyen algoritmayı tasarlayalım.



Dynamic Programming → 1950's Richard Bellman

Fibonacci Numbers

0 1 1 2 3 5 8 13

```
int fibonacci (int n) {
```

```
if (n == 0)
```

```
return 0;
```

if $(n == 1)$

```
return 1;
```

else

```
return fibonacci(n-1) + fibonacci(n-2);
```

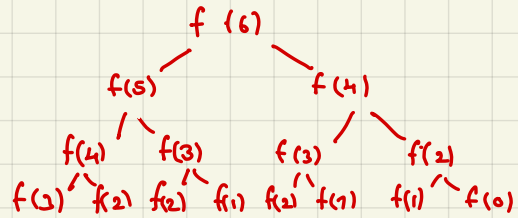
3



```
for (i=2; i ≤ n; i++)
```

$$\text{fib}_0[i] =$$
$$\text{fib}[i-1] +$$

```
fibonacci[i-2];
```



$$\text{fib}(n) = \begin{cases} \text{fib}(n-1) + \text{fib}(n-2) & \text{if } n > 1 \\ 1 & \text{if } n = 1 \\ 0 & \text{if } n = 0 \end{cases}$$

MINIMUM COIN CHANGE PROBLEM

coins[] = {1, 2, 5} value = 11

$$1 + 1 + 1 + \dots + 1 = 11$$

$$1+1+1+\dots 2 = 10$$

$$5 + 5 + 1 = 3$$

Bu adam girdi bütüdükçe

Sikanti

```
int minCoin(int Coins[], int n, int value)
```

3

```
int minCount = MAX;
```

```
if (value == 0)
```

```
return 0;
```

```
for (i=0; i < n; i++) {
```

```
if (coins[i] <= value) {
```

```
tmp = minCoin( Coins, n, value - coins[i]);
```

```
if (tmp < MAX) & (tmp + 1 < minCont)
```

```
minCount = trip + 1; }
```

3

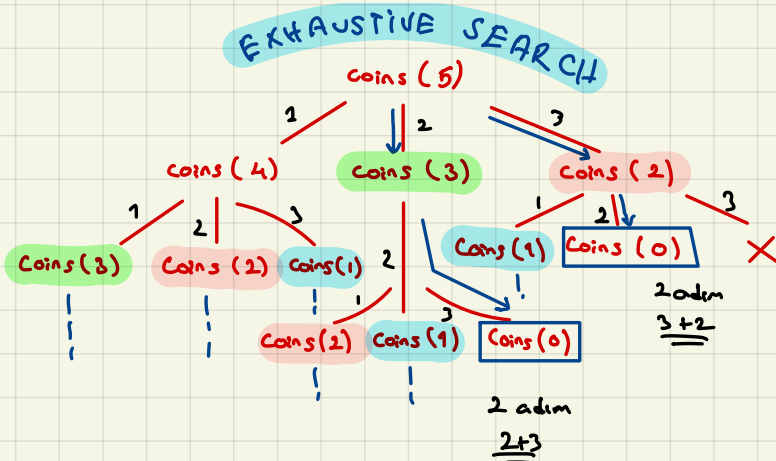
if (min Count == MAX)

```
return -1;
```

else

```
return minCount;
```

۳



SUB PROBLEMS ARE DEPENDENT!

MEMORIZATION

	0	1	2	3	4	5	6	7	8	9	10	11
0	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞

→ minimum # of coins for each amount

1 TL	0	1	2	3	4	5	6	7	8	9	10	11
1,2 TL	0	1	2	2	3	3	4	4	5	5	6	
1,2,5 TL	0	1	1	2	2	1	2	2	3	3	2	3

$C[P]$: minimum # of coins
X: first coin

$$C[P] = 1 + C[P-X]$$

$$C[P] = \begin{cases} \min \{ C[P-d_i] + 1 \} & \text{if } P > 0 \\ 0 & \text{if } P = 0 \end{cases}$$

↓
Param: [1,2,5] dizimisi

TRY ALL POSSIBLE X and take the minimum!

===

$$C[0] = 0$$

for p=1 to n do

{

$$\min = \infty$$

for i=1 to k

{ if $d[i] \leq P$ then

if $(1 + C[P-d[i]]) < \min$

then

$$\min = 1 + C[P-d[i]]$$

}

$$C[P] = \min$$

}



Karmosiklik $O(n \times k)$

$$7 TL = \min \begin{cases} i=1 & C[7-1] + 1 \\ i=2 & C[7-2] + 1 \\ i=3 & C[7-5] + 1 \end{cases}$$

↓
 $d[3] = 5$

0-1 Knapsack Problem

abandon select

W: max weight

n: # of items

ith item: value, weight

Brute force: 2^n exponential

		0	1	2	3	4	5
Item	0	0	0	0	0	0	0
1	0	0	12	12	12	12	12
2	0	10	12	22	22	22	22
3	0	10	12	22	30	32	32
4	0	10	15	25	30	37	37

item	weight	value
1	2	12
2	1	10
3	3	20
4	2	15

W=5kg

Case1: thief takes item i

$$P(i, w) = P(i-1, w-w_i) + v_i$$

Case2: thief doesn't take item i

$$P(i, w) = P(i-1, w)$$

not include item

before item i

$$w_i > w \quad P[5, 3] = P[4, 3]$$

isine weight [5]=7>3

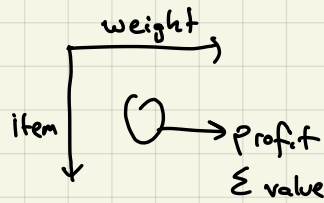
Take:

$$\max \{ v_i + P[i-1, w-w_i], P[i-1, w] \}$$

$$P(5, 11) = \max \{ 28 + 7, 40 \} \quad 40 \text{ not Taken}$$

item Value weight

1 → 1 → 1
2 → 6 → 2
3 → 18 → 5
4 → 22 → 6
5 → 28 → 7



$P[i, w]$
item weight

	0	1	2	3	4	5	6	7	8	9	10	11
0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	1	1	1	1	1	1	1	1	1	1	1
2	0	1	6	7	7	7	7	7	7	7	7	7
3	0	1	6	7	7	18	19	24	25	25	25	25
4	0	1	6	7	7	18	22	24	28	29	29	40
5	0	1	6	7	7	18	22	28	29	34	35	40

Karşıklık $\rightarrow O(n * w)$
of items # total weight

$i, w \rightarrow$ matrisin dizileri
 $w_i \rightarrow$ i'nci itemin weight i

Recurrence Relation

$$P[i, w] = \begin{cases} P[0, j] = 0, P[i, 0] = 0 & \text{if } i=0, j=0 \\ P[i-1, w] & \text{if } w_i > w \text{ not Taken} \\ \max \{ \underbrace{V_i + P[i-1, w-w_i]}_{\text{alır san}}, \underbrace{P[i-1, w]}_{\text{almaz san}} \} \end{cases}$$

i. satır
w. sütundaki
kazanç

Edit Distance (Değerli dil işleme için, iki kelimenin benzerliklerini ölçmek için)

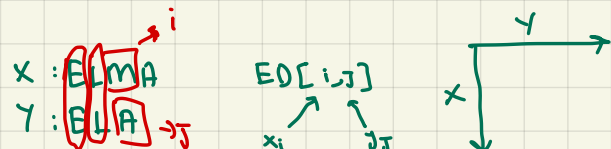
(Gen dizilimi, akrabalık testinde kullanılır) G A T A T
A G T A G

I N T E N T I O N
* E X E C U T I O N
↓ ↓ ↓ ↓ ↓
Delete Change Insert → distance 5

① substitute → some change (change $(x, i, a) \rightarrow x_1 \dots x_{i-1} \overset{x_i}{a} x_{i+1} \dots x_n$)

$x_i = y_j : ED[i, j] = ED[i-1, j-1]$
 $x_i \neq y_j : ED[i, j] = ED[i-1, j-1] + 1$

② Delete : $x_1, x_2, \dots, x_{i-1}, x_{i+1}, \dots, x_n$
 $x_i \neq y_j : ED[i, j] = ED[i-1, j] + 1$



③ Insert : $x_1, x_2, \dots, x_i, a, x_{i+1}, \dots, x_n$
 $x_i \neq y_j : ED[i, j] = ED[i, j-1] + 1$

Recurrence Relation

Edit Distance

$$ED[i, j] = \min \begin{cases} ED[i-1, j] + 1 & \text{: deletion} \\ ED[i, j-1] + 1 & \text{: insertion} \\ ED[i-1, j-1] + \text{cost} & \begin{cases} \text{if } x[i] = y[j] & \text{cost} = 0 \\ \text{if } x[i] \neq y[j] & \text{cost} = 1 \end{cases} \end{cases} \quad \text{: Substitution}$$

Initialization: $ED[i][0] = i$ $ED[0][j] = j$

	A	R	T	S	
M	0	1	2	3	4
A	1	1	2	3	4
T	2	1	2	3	4
H	3	2	2	3	3
S	4	3	3	3	3

ED(" ", "MATHS") = 5

M A T H S
A R T * S
↓ change ↓ change ↓ Delete

M A T H S
- A R T S
Delete ch. ch.

Levenshtein

Örnekler tabloda soldan geriye gelindiğinde oluşur

Longest Common Subsequence

Subsequence of word

yakamoz

↓
yak, az
yaz, kaz
ak, koz
⋮

→ ileri yönelik değil

Common subsequence

X: yakamoz Y: yazkaz

$LCS(X, Y) = \begin{cases} yaz \\ yaz \end{cases}$

Case 1: $x_i = y_j \rightarrow LCS[i, j] = LCS[i-1, j-1] + 1$

Case 2: $x_i \neq y_j \rightarrow \begin{cases} x \text{ is not part of CS} : LCS[i-1, j] \\ y \text{ is not part of CS} : LCS[i, j-1] \end{cases}$

Recurrence Relation

$$LCS[i, j] = \begin{cases} 0 & \text{if } i=0 \text{ or } j=0 \\ LCS[i-1, j-1] + 1 & \text{if } i>0, j>0 \text{ and } x_i = y_j \\ \max \{ LCS[i-1, j], LCS[i, j-1] \} & \text{if } i>0, j>0 \text{ and } x_i \neq y_j \end{cases}$$

$\uparrow$ x is not part of CS $\uparrow$ y is not part of CS

	B	D	C	B
B	0	0	0	0
A	0	1	1	1
C	0	1	1	2
D	0	1	2	2
B	0	1	2	3

LCS ("BDCB", "BADCB") = 3

BDCB

BADB

$X_i = Y_j \rightarrow B[i,j]: \text{ADD } XY$

$X_i \neq Y_j \rightarrow B[i-1,j]: \text{Skip } X$
 $B[i,j-1]: \text{Skip } Y$

$\text{cost}[i-1,j] > \text{cost}[i,j-1]$

$\text{cost}[i,j-1] > \text{cost}[i-1,j]$

int LCS (char * x, char * y, int m, int n, char * lcs_str) {

int L [m+1][n+1];

int i, j;

for (i=0; i<=m; i++) {
 for (j=0; j<=n; j++) {

L[i][j] = 0;

}

}

for (i=0; i<=m; i++) {

for (j=0; j<=n; j++) {

if (i=0 || j=0)

L[i][j] = 0;

else if (x[i-1] == y[j-1])

L[i][j] = L[i-1][j-1] + 1;

else

L[i][j] = max (L[i-1,j], L[i,j-1]);

}

}




```

i = m;
j = n;

int index = L[m][n];

lcs_str[index] = '\0';

while (i > 0 && j > 0)
{
    if (X[i-1] == Y[j-1])
    {
        lcs_str[index-1] = X[i-1];    i--;    j--;    index--;
    }
    else if (L[i-1, j] > L[i, j-1])
        i--;
    else
        j--;
}
return L[m][n];
}

```

```

int max ( int a, int b) {
    return (a > b) ? a : b;
}

```

```

int main ( )

```

```

{
    char X [MAX_LEN];
    char Y [MAX_LEN];
    char substring [MAX_LEN];

    printf (" Birinci Dizi: ");
    scanf ("%s", X);

    printf (" ikinci Dizi: ");
    scanf ("%s", Y);

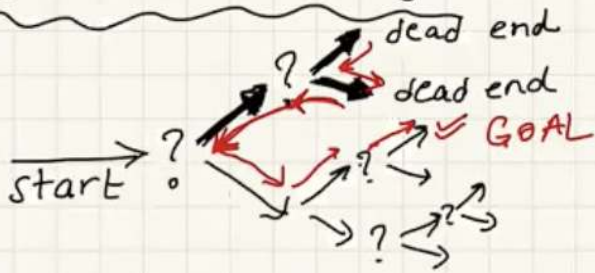
    int m = strlen(X);
    int n = strlen(Y);

    int lcs_result = LCS(X, Y, m, n, substring);

    return 0;
}

```

Backtracking



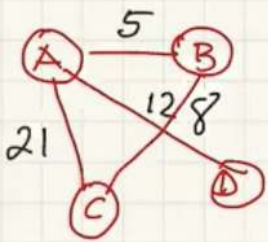
Depth First Search (DFS)

adjacency matrix
adjacency list

space
 $|V|^2$
 $|V| + |E|$

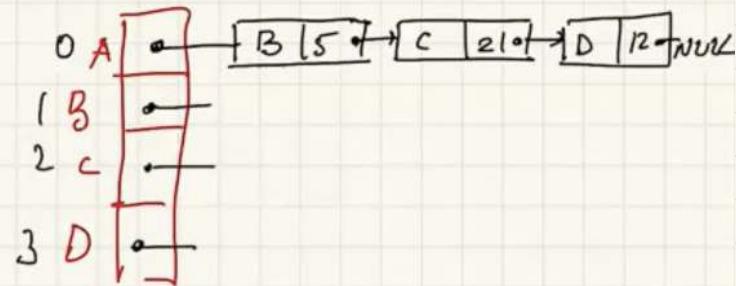
$O(\text{edge between } (v) \text{ and } (w))$

1
degree (v) degree(A) = 3



undirected graph

	0	1	2	3
A		5	21	12
B	5		8	
C	21	8		
D	12			

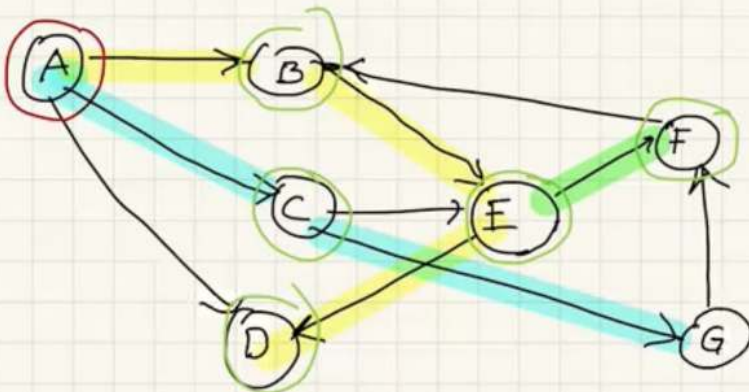


matrix $\xrightarrow{\text{ise}}$ adjacency list kullanılır

deneye $\xrightarrow{\text{ise}}$ adjacency matrix kullanılır

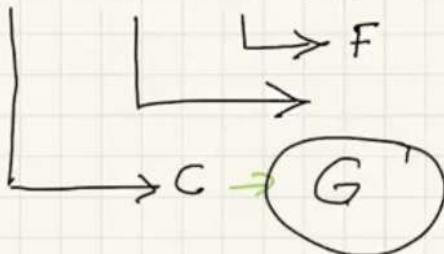
DFS → Depth First Search

A' dan G' ye yol var mı?



A → B → E → D →

A B E D F C G

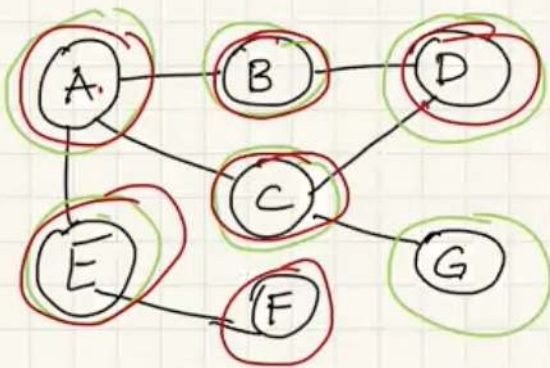


Depth first Search

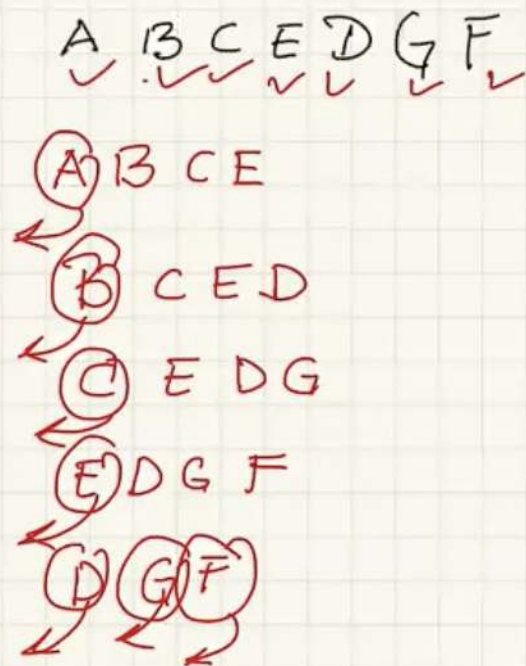
→ Stack ve Recursive olarak kullanılabilir.

- Topological Sort
- Find path from a to b
- cycle detection
- connected component

BFS Breadth First search = QUEUE



- Shortest path
- Bit torrent = peer to peer network
- Gps navigation system
- minimum spanning tree
- crawler



1. graf yapısı oluştur.

node →
in degree
ad sayı

2. eleme fonksiyonu

in degree $< m$

tekrar in degree hesapla

tekrar ele

while in degree $< m$

3. bağlantı sayısını hesapla

4. influencer tespiti yapan fonk

in degree $\geq x$

ise inf ✓

top. bağlantı sayısı $\geq y$

typedef struct node {

char * name;

int in-degree;

struct node * next;

} node;

void create_graph (node* nodes[], int num_nodes) { }

void eliminate_nodes (node* nodes[], int num_nodes) {

int i;

for (i = 0; i < num_nodes; i++) {

if (nodes[i] → in-degree $< m$) {

deletion

}

}


```
int eliminated = 1;
```

```
while (eliminated > 0) {
```

```
    eliminated = 0;
```

```
    for (i = 0; i < num_nodes; i++) {
```

```
        if (nodes[i] → in-degree < m) {
```

```
            deletion, eliminated++;
```

```
        }
```

```
    }
```

```
}
```

```
int total_incoming_connections (node* nodes[], int num_nodes, node* target)
```

```
{
```

```
    int i;
```

```
    int total = 0;
```

```
    for (i = 0; i < num_nodes; i++) {
```

```
        if (nodes[i] → next == target)
```

```
            total++;
```

```
    }
```

```
int visited [num_nodes];
```

```
for (i = 0; i < num_nodes; i++) {
```

```
    visited[i] = 0;
```

```
}
```

```
total += dfs (nodes, num_nodes, target, visited);
```

```
return total;
```

```
int dfs (node* nodes[], int num_nodes, node* target, int visited[]) {
```

```
    int i;
```

```
    int total = 0;
```

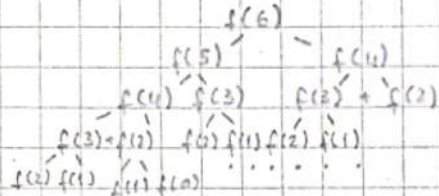
Dynamic Programming → 1950's Richard Bellman

→ Fibonacci Numbers

0 1 1 2 3 5 8 13 ...

```
int fibonacci (int n) {
    if (n == 0)
        return 0;
    if (n == 1)
        return 1;
    else
        return fibonacci(n-1) + fibonacci(n-2);
}
```

bazı elemanlar birden fazla kez hesaplanıyor.



```
for (i = 2; i < n; i++)
    fibonacci[i] = fibonacci[i-1] + fibonacci[i-2];
```

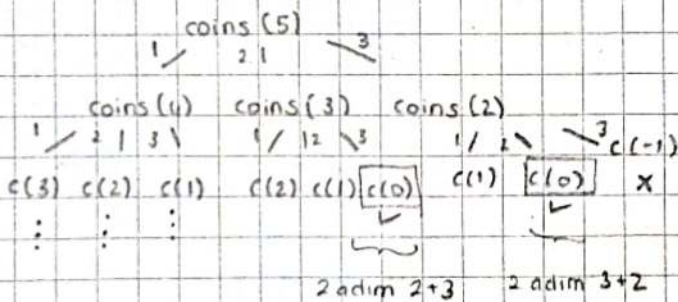
→ Minimum Coin Change Problem

coins[] = {1, 2, 5} value = 11 Answer = 5 + 5 + 1

1+1+1...+1 → 11
1+1+1...+2 → 10
⋮
5+5+1 → 3

Exhaustive Search

coins[] = {1, 2, 3} value = 5 olsun (daha küçük değerler)



→ yine aynı şeyler birden fazla hesaplanıyor.

→ Subproblems are dependent!

exponential brute force

$O(n \cdot \text{value})$

```
int minCoin (int coins[], int n, int value) {
```

```
    int minCount = MAX;
```

```
    if (value == 0)
```

```
        return 0;
```

```
    for (i = 0; i < n; i++)
```

```
        if (coins[i] <= value) {
```

```
            tmp = minCoin (coins, n, value - coins[i]);
```

```
            if (tmp < MAX) && (tmp + 1 < minCount)
```

```
                minCount = tmp + 1;
```

```
        }
```

```
    }
```

if (minCount == MAX)

return -1;

else

return minCount;

}

return minCount;

→ hesaplama bilgileri bir yerde tutarak kullanma → memorization

0-2 TL
min(0, 1, 2) → 1+1+1
coins = {1, 2, 5} value = 11 answer 5+5+1 → 3 adet

Amount

Amount	0	1	2	3	4	5	6	7	8	9	10	11
0 TL →	0	1	2	3	4	5	6	7	8	9	10	11
1, 2 TL →	0	1	1	2	2	3	3	4	4	5	5	6
1, 2, 5 TL →	0	1	1	2	2	1	2	2	3	3	2	3

→ minimum number of coins for each amount

5 TL → min(6, 2+1, 3+1) → 2 geri giderek
5 geri giderek

5 lirayı, etkilemez

$c[p]$: min number of coins
X: first coin
 $c[p] = 1 + c[p-X]$
X TL coin

$$c[p] = \begin{cases} \min \{c[p-d_i] + 1\} & \text{if } p > 0 \\ & i = d_i \leq p \\ \emptyset & \text{if } p = 0 \end{cases} \rightarrow c[8] = \min \begin{cases} c[8-1] \\ c[8-2] \\ c[8-5] \end{cases}$$

TRY ALL POSSIBLE X and take the minimum!

```

c[0] = 0
for p = 1 to n do
    min = ∞
    for i = 1 to k
        if d[i] ≤ p then
            if (1 + c[p-d[i]]) < min then
                min = 1 + c[p-d[i]]
    }
}

```

0-1 Knapsack Problem

abandon → select

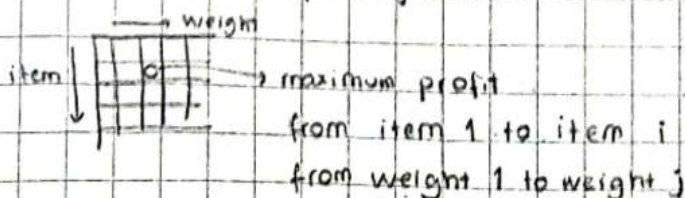
W: max. weight

n: # of items

ith item = value (v_i), weight (w_i)

Brute Force: 2ⁿ // exponential

maximize $\sum v_i$ subject to W



item	weight	value
1	2	12
2	1	10
3	3	20
4	2	15

W=5 kg

item	weight	0	1	2	3	4	5
0		0	0	0	0	0	0
1		0	0	12	12	12	12
2		0	10	12	22	22	22
3		0	10	12	22	30	32
4		0	10	15	25	30	37

max degeri koru

case 1: thief takes item i

$$p(i, w) = p(i-1, w-w_i) + v_i$$

case 2: thief does NOT take item i

$$p(i, w) = p(i-1, w)$$

not include item i

before item i

p(i, w)
profit

$w_i > w$ ise alınan yük cantanın değerini aşıyorsa

$\max \{ v_i + p(i-1, w-w_i), p(i-1, w) \}$, max değerleri karşılaştırılır

Hangi yükleri aldığımızı bulurken NOT TAKEN , TAKEN , 4, 2, 1

üstündeki değerle karşılaştır.

değer değişmemişse o alınmaz, değer değişmişse o alınır

alınan itemin ağırlığı kadar geri git

0'a ulaşana kadar devam

Karşılaştığı $\rightarrow O(n \cdot w)$ $n \rightarrow$ item no $w \rightarrow$ total weight

Recurrence Relation

$$p[i, w] = \begin{cases} p[0, j] = 0, p[i, 0] = 0 & \text{if } i=0, j=0 \\ p[i-1, w] & \text{if } w_i > w \text{ NOT TAKEN} \\ \max \{ v_i + p[i-1, w-w_i], p[i-1, w] \} & \end{cases}$$

1. satır w sütündeki boşluk

alınmam

almazsam

Edit Distance

1) grappe $\rightarrow$ Dictionary graph, graft, grail, giraffe

2) Computational Biology

ACGTGGA

AGTAGA

I N T E * N T I O N
E X E C U T I O N
distance 5

$x = x_1 \dots x_n$

$y = y_1 \dots y_n$

ED[i, j]
 x_i y_j

X: ELMA
Y: ELA

① Substitute $\begin{cases} \text{same} \\ \text{change} \end{cases}$
 $\text{change}(x, a) \rightarrow x_1 \dots x_{i-1} a x_{i+1} \dots x_n$

$$x_i = y_j : ED[i, j] = ED[i-1, j-1]$$

$$x_i \neq y_j : ED[i, j] = ED[i-1, j-1] + 1$$

② Delete: $x_1 x_2 \dots x_{i-1} \mid x_{i+1} \dots x_n$
 $x_i \neq y_j : ED[i, j] = ED[i-1, j] + 1$

③ Insert: $x_1 x_2 \dots x_i \mid x_{i+1} \dots x_n$
 $x_i \neq y_j : ED[i, j] = ED[i, j-1] + 1$

$$ED[i,j] = \min \begin{cases} ED[i-1][j] + 1 & : \text{deletion} \\ ED[i][j-1] + 1 & : \text{insertion} \\ ED[i-1, j-1] + \text{cost} & \begin{cases} \text{if } x[i] = y[j] & \text{cost} = 0 \\ \text{if } x[i] \neq y[j] & \text{cost} = 1 \end{cases} : \text{substitution!} \end{cases}$$

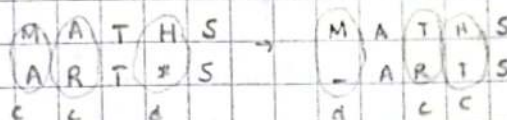
Initialization $ED[i][0] = i$ $ED[0][j] = j$

X: MATHS Y: ARTS

	A	R	T	S
M	1	1	2	3
A	2	1	2	3
T	3	2	2	3
H	4	3	3	3
S	5	4	4	3

$ED("", "MATHS") = 5$

time comp: $O(n \cdot m) + N^2$



→ min. distance

Longest Common Subsequence

subsequence of a word

→ yakamoz → yak, yaz, ak, az, kaz, koz ... ~~ayak~~ → geri gidilmeden

common subsequence

yakamoz yazgaz
yaz ← yaz
yaz

Brute Force $\begin{matrix} x[1..n] \\ y[1..m] \end{matrix} \} O(n \cdot 2^m)$

DNA ALPHABET (A, C, G, T)

X: ATCTGAT } $LCS(X, Y) = \begin{cases} TCTA \\ TGAT \end{cases}$
Y: TGCATA

$LCS[i, j]$

$x[i]$ ve $y[j]$ ye kadar LCS değeri

case 1: $x_i = y_j \rightarrow LCS[i, j] = LCS[i-1, j-1] + 1$ → ortak bir harf daha bulundu

case 2: $x_i \neq y_j$ → x_i is not part of cs: $LCS[i-1, j]$

→ y_j is not part of cs: $LCS[i, j-1]$

$$LCS(i, j) = \begin{cases} 0 & \text{if } i=0, j=0 \\ LCS[i-1, j-1] + 1 & \text{if } i>0, j>0 \text{ and } x_i = y_j \\ \max \{ \underset{\substack{\uparrow \\ x \text{ not part} \\ \text{of LCS}}}{LCS[i-1, j]}, \underset{\substack{\uparrow \\ y \text{ not part} \\ \text{of LCS}}}{LCS[i, j-1]} \} & \text{if } i>0, j>0 \text{ and } x_i \neq y_j \end{cases}$$

X: "BACDB" Y: "BDCB"

	B	A	C	D	B
B	0	0	0	0	0
A	0	1	1	1	1
C	0	1	1	1	1
D	0	1	2	2	2
B	0	1	2	3	3

BCB $\rightarrow$ LCS("BACDB", "BDCB") = 3
BDB

SLATI

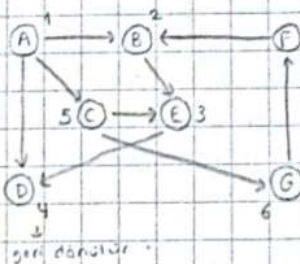
Graph Traversal - Depth First Search (DFS)

DFS (visit a vertex s)

Mark s as visited

Visit all unmarked vertices v adjacent to s.

↑
recursive



A'dan G'ye yol var mı?

A $\rightarrow$ B $\rightarrow$ E $\rightarrow$ D

$\hookrightarrow$ C $\rightarrow$ G

A	B	C	D	E	F	G	visited
0	0	0	0	0	0	0	
1	1	1	1	1	1	1	
							checked

1) recursive DFS

graph initial point
DFS(G, s)

mark s as visited

for all neighbors w of s in Graph G

if w is not visited

DFS(G, w)

}

2) non-recursive DFS $\rightarrow$ using stack

DFS(G, s)

stack.push(s)

mark s as visited

while (stack is not empty)

v = stack.pop()

for all neighbors w of v in Graph G

if w is not visited

stack.push(w)

mark w as visited

}

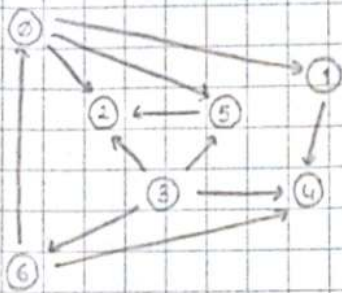
}

}

}

Topological Sort - DFS

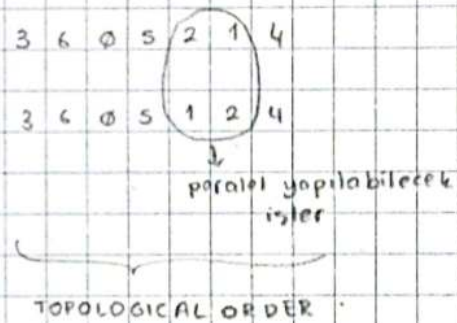
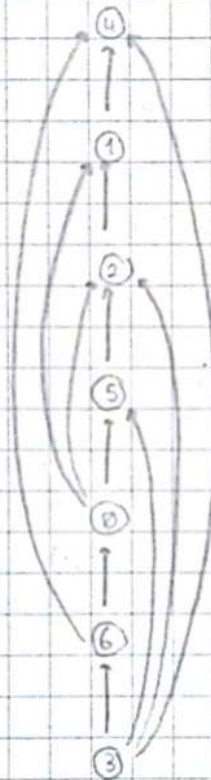
Directed Acyclic Graph (DAG)



in-degree	Adjacency List
1	0 → 1 → 2 → 5
1	1 → 4
3	2 → x
3	3 → 2 → 4 → 5 → 6
3	4 → x
2	5 → 2
1	6 → 0 → 4

dfs(0)
 dfs(1)
 dfs(4)
 [4 done]
 [1 done]
 dfs(2)
 [2 done]
 dfs(5)
 check-2
 [5 done]
 [0 done]
 dfs(3)
 check 2
 check 4
 check 5
 dfs(6)
 check 0
 check 4
 [6 done]
 [3 done]

reverse list



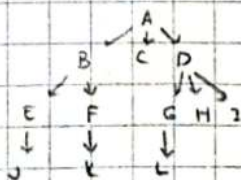
Graph Traversal - Breadth First Search (BFS)

BFS (from source vertex s)

Put s onto a FIFO queue

Repeat until the queue is empty.

- remove the least recently added vertex v.
- add each of v's unvisited neighbors to the queue and mark them as visited



BFS
 A B C D E F G H I J K L

BFS (G, s) {

 Queue_enqueue (s)

 mark s as visited

 while (Queue is not empty) {

 v = Queue.dequeue ();

 for all neighbors w of v in G {

 if w is not visited {

 Queue_enqueue (w)

 mark w as visited

 }

 }

 }